# DiracX developments

## Directions

**Alexandre F. Boyer** · CERN

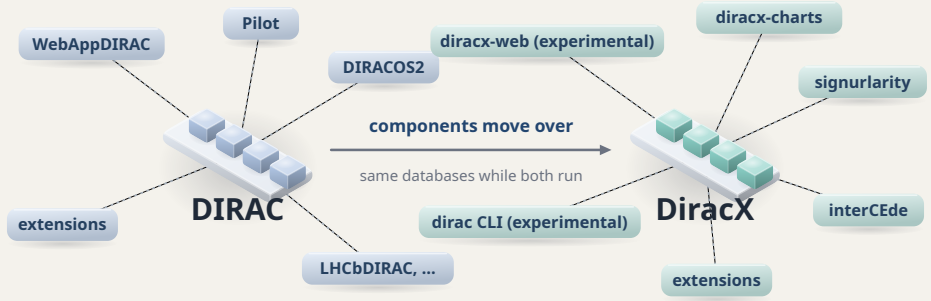
12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



# 1 · Where we are now

DIRAC and DiracX today

# Maintaining DIRAC, progressing on DiracX

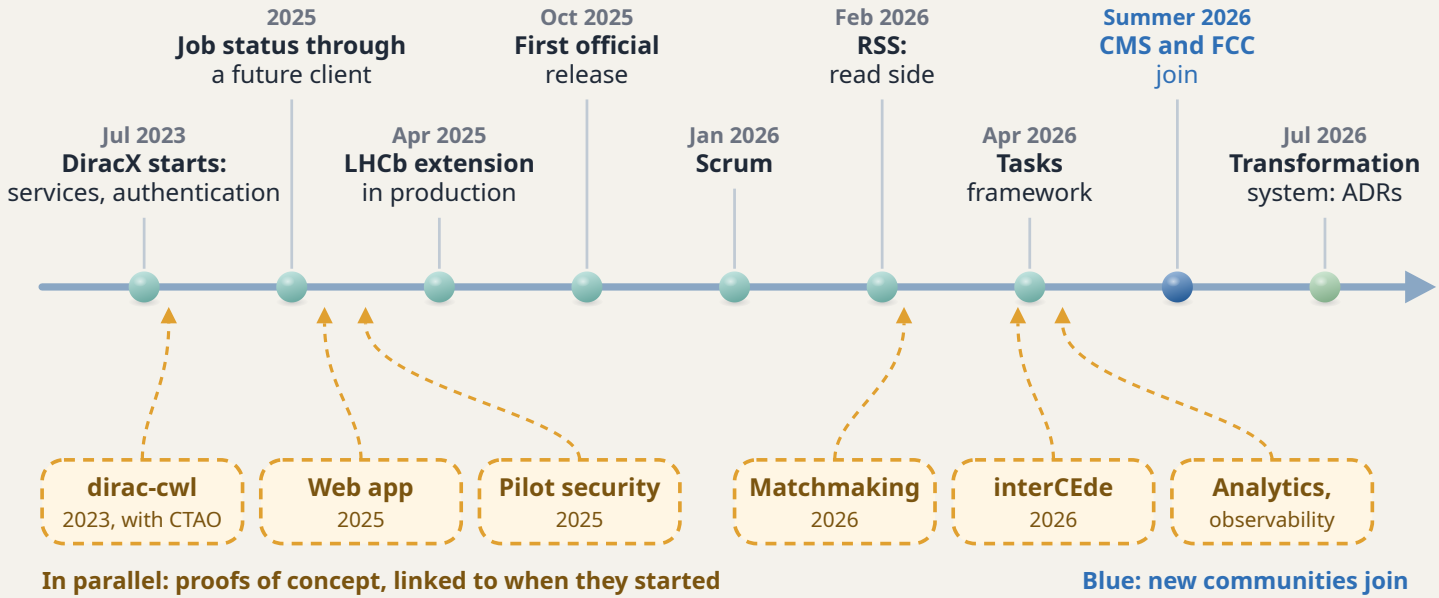



**DIRAC** v9.1.21 · v9.0.27 · v8.0.85
WebAppDIRAC 6.0.7 · DIRACOS2 2.63
**Maintained**: security fixes, and performance fixes when needed.

**DiracX** v0.6.1
diracx-charts 1.1.17
**Developed**, one component at a time.
The web app (diracx-web 0.1.0-a11) and the `dirac` CLI are still experimental.

> **Adopt what already exists in DiracX.** DIRAC is only patched, so that DiracX can progress.

# The journey so far




- **From 2023, three hackathons a year** onboarded new contributors, with no dashboard or regular meeting to follow progress. **Scrum** brought both in 2026.
- **Proofs of concept run in parallel**, not mandatory: we try an idea before committing to it.

# Many part-time hands




- **36 people** contributed since the last workshop, **10 to 18** in any given month, almost all part-time: about **5 to 6 full-time equivalents** in total, LHCb core developers included.
- They maintain DIRAC and develop DiracX at the same time.

> **Finding the best way to work with each other is paramount.**

# Delivered since the last workshop

### DiracX, in use

- From alpha to **23 releases**, the latest **v0.6.1** on 9 October 2026.
- **LHCb in production**; two other installations certifying.

### Usable now

**Sandboxes in S3**, **job status** through the future client, **resource status** read in DiracX, the **tasks framework**, and experimental **OpenTelemetry** observability.

### DIRAC kept safe

v8, v9.0 and v9.1 patched: **security fixes** and **SQL hardening**, with GridPP.

# 2 · Evolving requirements

New communities and new use cases

# What DIRAC is for, and what it values

> Running large-scale workflows of embarrassingly parallel jobs on heterogeneous, distributed computing and storage resources: high-throughput computing.

### Minimal dependencies

A database and a search index, on a few hosts.

### Multi-VO installations

One installation serves several communities.

### Extensible

Communities add their own pieces.

My own reading of what communities have asked for over the last few years.

# Requirements keep evolving

### New communities

**CMS and FCC**, who joined this summer, bring new requirements. CMS has no DIRAC installation and runs its own workload management: Wednesday's CMS talk.

### Advanced needs, in extensions

**CTAO, LHCb, Belle II and JUNO** built extensions for advanced needs that are now close to each other. They belong in the common project: CTAO's use of **CWL** turned out useful for DiracX as a whole.

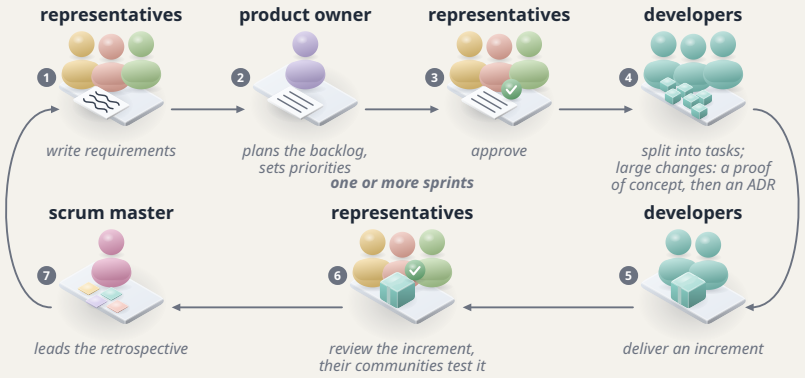
**These advanced needs do not fit today's data model**, shared by DIRAC and DiracX, and DIRAC must stay as stable as possible: they need a new model, decoupled from DIRAC.

**DOps is where you share your use cases.** The representatives of the communities decide together whether the vanilla project will fulfil them, or say clearly that it will not. Bring them early: someone else may need the same thing.

# 3 · Adapting quickly

Responding to changing requirements, in small steps

# Almost the current process



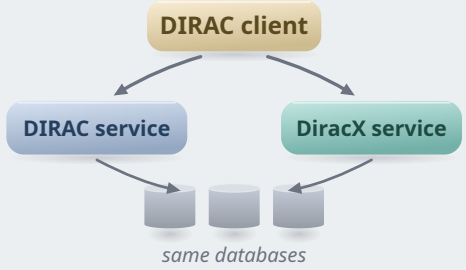

- **Scrum since 21 January 2026**: two-week sprints; planning and refinement in one weekly meeting; a monthly meeting with the representatives (DOps); hackathons.
- **Large changes start with a proof of concept, then an ADR** that the other developers approve. ADRs and roadmap are public; the board is not.

> **Individuals and interactions over processes and tools.** The process is a guide, not a rule, and it adapts as we learn.

# Not a port: a reimplementation

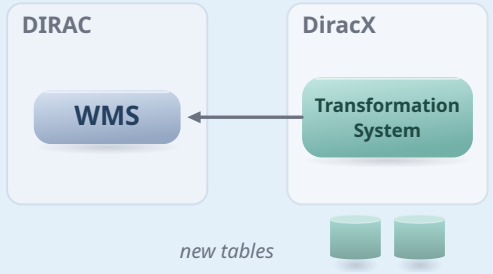
## Future clients

**DMS**: a DIRAC client calls DiracX instead of DIRAC, over the same databases. **RSS**: the read side is in DiracX; DIRAC's adapter is in review.




## A new data model

**The spine**: workgraph, transformations, parcels. New requirements need new tables and new logic. DIRAC's WMS is used as it is, as a backend.




## Standard protocols

**Systems internal to DIRAC give way to standards**: monitoring becomes OpenTelemetry.




This week: Data Management for DiracX · RSS in DIRAC and DiracX (all Wednesday)

# Standard protocols: one project, many contexts

- **We choose our tooling**: standard implementations work, not one product we have to maintain.
- **You connect DiracX to what you already run**: your identity provider, your storage, your monitoring, your databases.
- **Small or large**: a small installation does not need a cutting-edge stack; a large one plugs in its own.

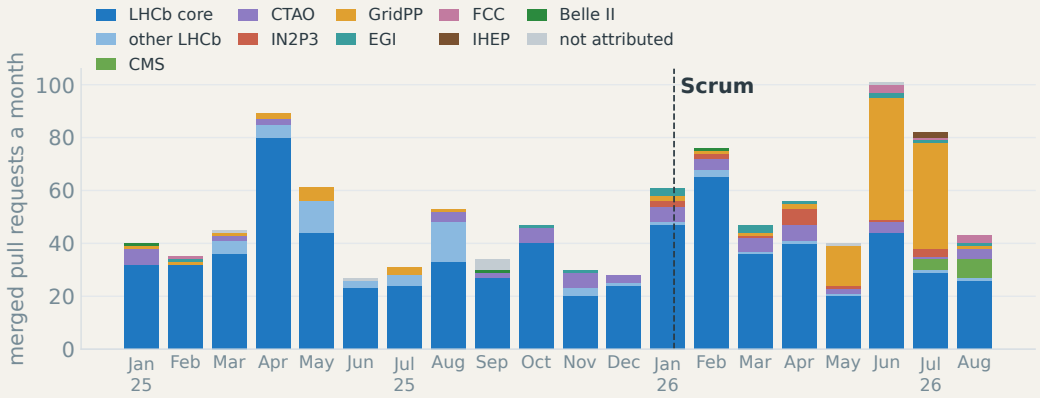
| FOR | PROTOCOL | IN THE CHARTS | OR YOURS |
|---|---|---|---|
| identity | **OIDC** | Dex; INDIGO IAM optional | your SSO or IAM |
| sandboxes | **S3** | SeaweedFS, optional | your site's S3 service |
| metrics, traces | **OpenTelemetry** | collector, Jaeger, Grafana, optional | your monitoring |
| state, job parameters | **SQL, OpenSearch** | MySQL, OpenSearch: **tests and demos only** | your IT's databases: **required in production** |

**We ship a minimal option for each, so DiracX works out of the box, as DIRAC does.** Databases are the exception: running MySQL or OpenSearch from the charts is not supported in production. Running without Kubernetes: Thursday's talk.

# 4 · How we work, measured

What the numbers tell us about our process

# More hands, from more communities




**634 pull requests** merged across all DIRACGrid repositories since the last workshop (367 DIRAC, 261 DiracX), from nine communities; **17 new contributors**.

Outside the five core developers: **36% of merged PRs, 28% of issues opened, 53% of story points.**

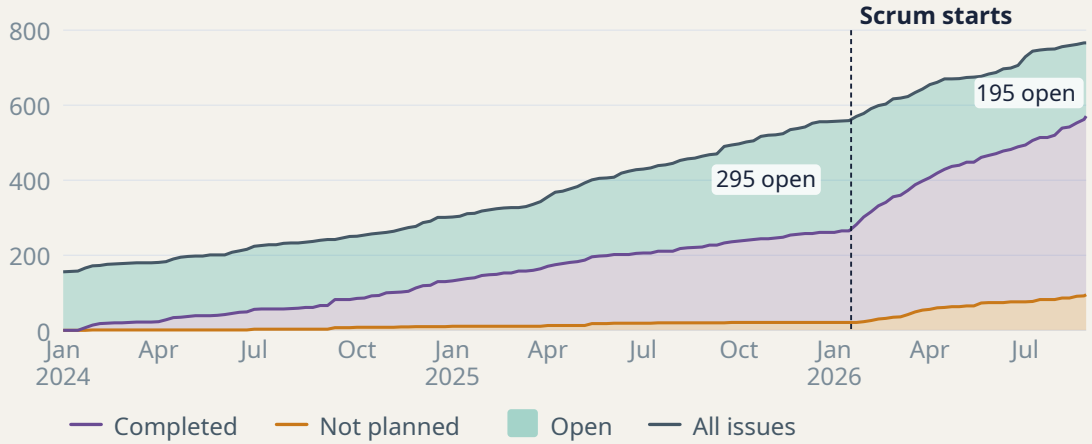
Read with care: about half of the 2026 rise in PRs is **one GridPP developer** hardening DIRAC's SQL. CMS merged 14 changes since July.

# Since Scrum: the backlog is shrinking

**295 → 195**
open issues, January 2026 to September

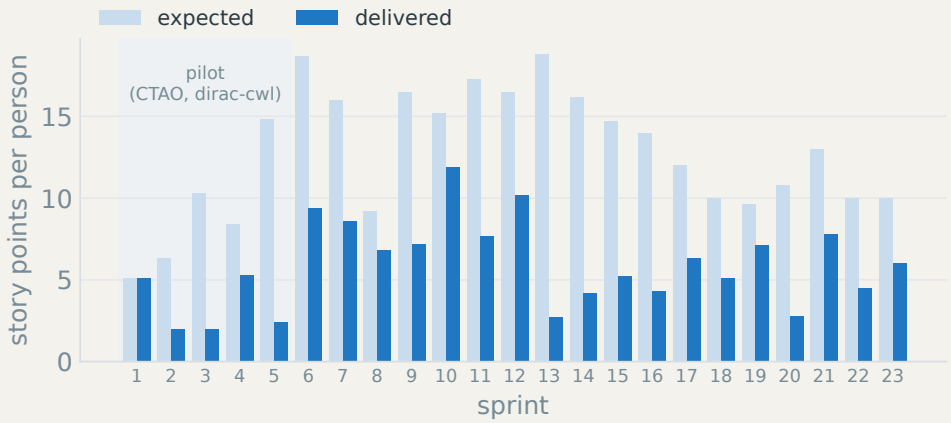
**10.6 → 36.6**
closures a month; 26.6 completed, 10.0 not planned or duplicate

**16.3 → 27.3**
issues opened a month




All issues in the DIRACGrid repositories, bots excluded. Part of the fall is triage, and about 120 of the 195 still open are not triaged yet.

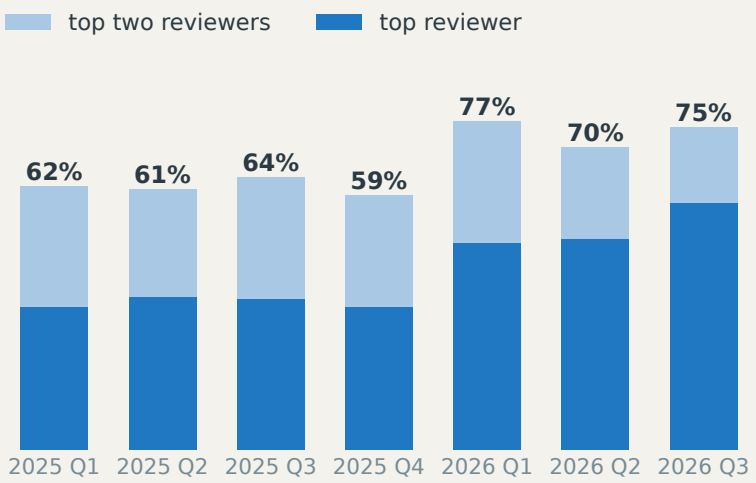
# We plan for twice what we deliver




- Since January, a median of **14 story points** per person and sprint expected, **6.6 delivered**. No sprint delivered what was expected.
- Delivered velocity follows availability: workshops, conferences, holidays.
- **Unplanned work lives in DIRAC**: 56% of the story points delivered on DIRAC were unplanned, against 11% on DiracX.

**For developers:** we overestimate what a sprint can hold, partly because we do not count the unplanned DIRAC maintenance. Plan for it.

# The bottleneck is review




Share of first reviews, by quarter

- **Two people give three quarters of first reviews.** None of the 14 contributors new since January has given one yet.
- **Smaller pull requests get reviewed sooner**: 2.6 hours under 10 lines, 23 hours from 500 lines. On small changes the core five wait 2.5 hours, everyone else 11.
- Design review (ADRs, in meetings and on chat) is not in these numbers.

**Question for developers:** kanban style, should you review before starting something new once your tasks are done?

How to split a large change into stacked pull requests

# Where work waits




— **Specification and review are the narrow parts**, and they will matter more with every new component.

# 5 · What’s next

How we could improve, and the roadmap

# Cheaper code: easier to explore, same bottlenecks

> *"AI's primary role in software development is that of an amplifier."*

DORA 2025 State of AI-assisted Software Development

- Writing code got cheap, so **spikes and proofs of concept** are easier to try.
- **Deciding what to build and checking it** did not get cheaper.
- It does **not change our process**. It shows its weak points sooner.

**Each one maps to what we saw**

- **Vague specification** → verbose, AI-generated issues that go stale. A clear ADR with a scenario test is what an agent, or a newcomer, can implement.
- **Review bottleneck** → more code means more to read, by the same two people.
- **Unplanned work** → faster code does not shrink urgent maintenance.
- **A new kind of user** → DiracX speaks HTTP and OAuth2, so an agent can use it through MCP. IHEP already operates DIRAC with an agent and MCP: Wednesday's talk.

→ *invest in* **specifications, tests and review**, *not in typing*

# Next: building the transformation system

> **Implementation follows the ADRs.** The technical details are in the next talk.

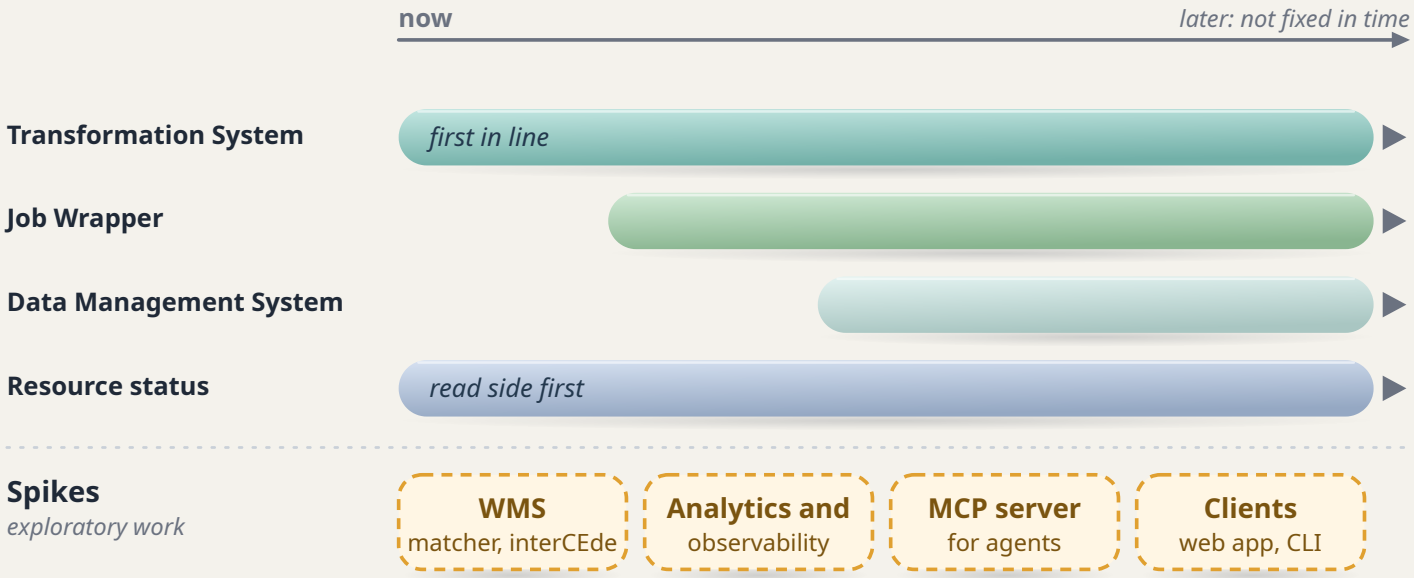
### Where it comes from

- **Use cases from the communities**: CMS, CTAO, FCC, Belle II and LHCb.
- **A design that comes from a proof of concept**, written down as ADRs.

### How it will be delivered

- **Increments you can test**, at DOps.
- **The more time people can give to the product**, and the better organised we are, the sooner.

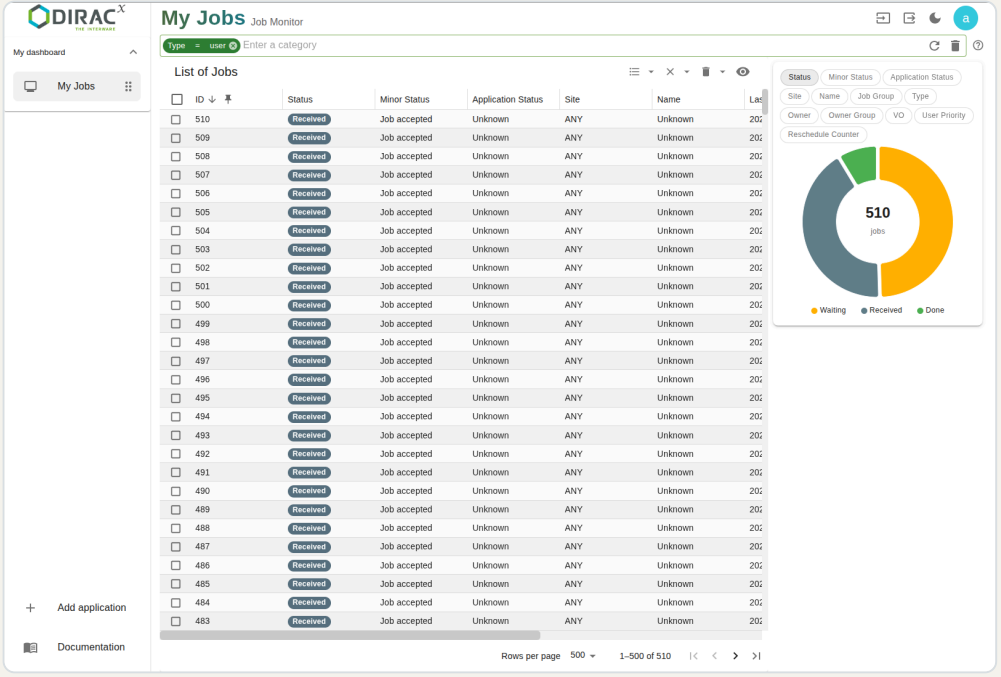
# The roadmap moves with the communities




*Priorities, not commitments. New requirements can reorder this page, and the team writes the detailed plan.*

# A word on the web app

Announced at past workshops, not on this week's agenda.



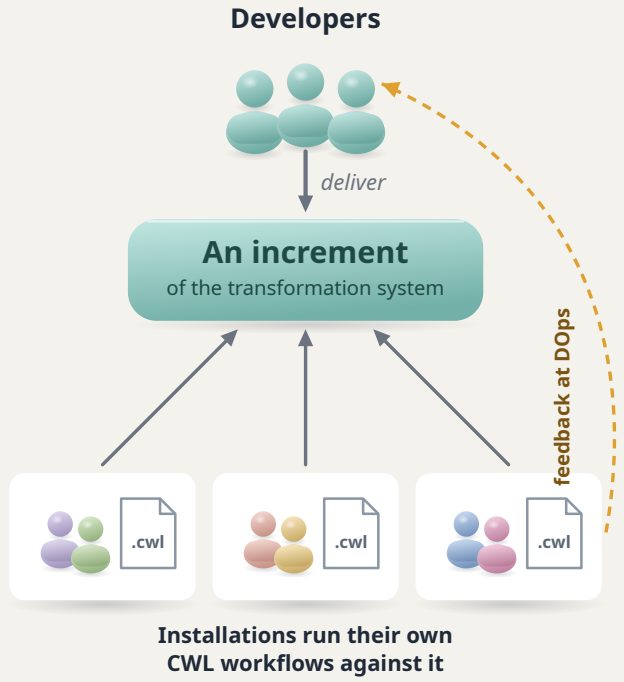

- **Today: a job monitor**, smooth and responsive; extensible, unit and end-to-end tested.
- **We promised a simple app on popular libraries.** AI changes the game: richer interfaces from simple components, fewer dependencies.
- **Next: a consistent, user-centric experience** around the new transformation system and its components.

> **Increments at DOps** once the transformation system is usable. Feedback welcome any time.

# If you run an installation today

## What you can count on

- **DIRAC keeps running**, with security patches and performance fixes when needed.
- **You choose when to switch each component**, usually at a shutdown; DIRAC and DiracX run in parallel meanwhile.
- **No data migration** for anything but the transformation system, which has its own tables.
- **Your extension keeps working** on DIRAC while you move.
- **Increments to test**, shown at DOps whenever a sprint delivers one. You test them with your own workflows in **CWL**; we give guidance on writing them.



# Take-aways

FOR INSTALLATIONS

- **Adopt what exists; DIRAC is only patched.** You move one component at a time, and the transformation system runs in parallel.

FOR REPRESENTATIVES AND DEVELOPERS

- **Specification and review are the bottlenecks.** Increments you can test, shown at DOps, keep us close to your needs.

A QUESTION FOR REPRESENTATIVES

- **At DOps:** is DOps enough for you to follow the work and agree on priorities? How should we collect your requirements?

FOR THIS AFTERNOON

- **Join the hackathon**: your workflows in CWL from 14:00, then the transformation system from 16:10.

# Thank you

Questions and comments welcome, this week and after.

**Alexandre F. Boyer** · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

